MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 7

Operating Systems

Three Small Programs: Processes, Redirection, a Pipe

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Environment and evidence 3

2.2 Part 1 - fork, exec and wait 3

2.3 Part 2 - A zombie, on purpose 4

2.4 Part 3 - Output redirection 5

2.5 Part 4 - A pipe 5

2.6 Optional - Three stages 6

3 Conclusions **7**

A Submitted source code **8**

A.1 run.c 8

A.2 zombie.c 9

A.3 redirect.c 9

A.4 pipeline.c 10

A.5 pipeline3.c 11

1 Introduction

Use small Linux C programs to demonstrate process creation, execution, reaping, descriptor redirection and interprocess pipes.

This report follows the teacher's handout and includes every required Observe answer. Terminal-style listings replace new screenshots. Recorded output is identified with its execution environment; shortened excerpts are marked. Source files and build instructions are provided in the submission directory.

2 Tasks and results

2.1 Environment and evidence

Built and tested on 23 October 2026 in Debian GNU/Linux, ARM64, as non-root user `student`. The Makefile builds with C11, `-Wall -Wextra -Werror`. The terminal blocks below come from actual runs. PIDs, dates, file listings and directory counts vary across machines. Complete captured output is in `terminal-output.txt`. The programs were tested in a temporary Linux container; they can also be compiled with the individual gcc commands from the handout.

```
make
# Or, for example:
gcc -Wall -Wextra -o run run.c
```

2.2 Part 1 - fork, exec and wait

`run.c` preserves the supplied lab program: the parent prints and flushes its PID, forks, the child prints its PID and parent PID, then uses `execlp` to become `ls -l`. The parent waits with `waitpid` and decodes the child's termination status. The explicit flush before fork prevents a buffered parent message from being duplicated when stdout is captured.

The two required one-line variants are kept as separate source files, `run_date.c` and `run_missing.c`, so all three behaviors remain reproducible without editing the final working `run.c`.

Actual `run` output, with the directory listing shortened:

```
$ ./run
parent: my PID is 8781
child:  my PID is 8782, my parent is 8781
total 536
[directory entries omitted]
parent: child 8782 ended with status 0
```

```
$ ./run_date
parent: my PID is 8783
child:  my PID is 8784, my parent is 8783
Fri Oct 23 09:38:56 UTC 2026
parent: child 8784 ended with status 0
$ ./run_missing
parent: my PID is 8785
child:  my PID is 8786, my parent is 8785
```

```
exec: No such file or directory
parent: child 8786 ended with status 127
```

2.2.1 1. Why perror runs only after a failed exec

A successful `execlp` replaces the child's process image and never returns to the following line. The same PID now runs the new program. If exec fails, it returns -1 and sets errno; only then does `perror("exec")` execute. The missing-program variant deliberately exits with 127 after printing its error.

2.2.2 2. Whose PID is printed as my parent

The child prints 8781 as its parent in the recorded `run` experiment, the PID printed by the original parent process. `getpid()` returns the calling process's own ID; `getppid()` returns its current parent's ID. Here the parent stays alive and waits, so the child is not reparented during this observation.

2.2.3 3. Exit statuses

The parent reports 0 for successful `ls` and `date`, and 127 for `nosuchprog`. These are the child's exit statuses. The supplied parent program itself returns 0 after reporting them, including the missing-program experiment; this differs from the child status it prints. Linux waitpid returns an encoded wait status, so `WEXITSTATUS` extracts the exit value when the child exited normally. A general shell would also handle signal termination using `WIFEXITED`/`WIFSIGNALED`.

2.3 Part 2 - A zombie, on purpose

`zombie.c` is the `ls` version of `run.c` with `sleep(20)` immediately before `waitpid`. The test started it in the background, inspected its child after two seconds, then waited for the parent to finish.

```
$ ps -o pid,ppid,stat,cmd -p 8803,8804
PID  PPID STAT CMD
 8803  8776 S    ./zombie
 8804  8803 Z    [ls] <defunct>
$ ps -o pid,ppid,stat,cmd -p 8804
PID  PPID STAT CMD
```

2.3.1 1. The Z state

The child's STAT letter is `Z`, meaning zombie: `ls` has exited, but the kernel retains its PID and termination status until its parent reaps it. It is not running useful code or sleeping to finish its work. The parent's `S` means sleeping.

2.3.2 2. What removes the zombie

After the parent wakes from its 20-second sleep, `waitpid(pid, &status, 0)` consumes the child's recorded exit information and removes its process-table entry. The second `ps` has only the header for that PID. The elapsed sleep alone does not reap a child; the wait call does.

2.4 Part 3 - Output redirection

`redirect.c` forks, closes descriptor 1 in its child, opens `out.txt` with `O_WRONLY | O_CREAT | O_TRUNC` and mode 0644, then execs `ls -l`. The parent retains its terminal/capture stdout and waits. The program checks fork/open/exec/wait errors and reports the child result.

```
$ ./redirect
open gave descriptor 1
done: the listing is in out.txt
```

`out.txt` was also checked to begin with the `total` header of a long directory listing. It is generated when the program runs, rather than submitted as a machine-specific listing.

2.4.1 1. Why open returned 1

`open` chooses the lowest unused descriptor. Standard descriptors 0, 1 and 2 were initially open; closing 1 made it the first free slot. The recorded diagnostic confirms that open returned 1. This experiment relies on the normal standard-descriptor setup.

2.4.2 2. Why unchanged ls writes to the file

`ls` still writes to descriptor 1. The forked child's descriptor table was modified before exec, and descriptors remain open across exec unless marked close-on-exec. Consequently `ls` inherits stdout pointing at `out.txt`. The diagnostic is sent to descriptor 2 and remains visible; the parent's final message also remains visible because descriptor changes were made only in the child.

2.4.3 3. Redirecting errors

For `2> err.txt`, close descriptor 2 before opening the error file, leaving descriptors 0 and 1 open so the file gets 2. A general implementation can use `dup2(fd, STDERR_FILENO)` to select the target explicitly. Descriptor 2 is standard error; descriptor 1 is standard output.

2.5 Part 4 - A pipe

`pipeline.c` creates a pipe and forks two children. The producer closes stdout and duplicates the write end onto descriptor 1, then execs `ls /etc`. The consumer closes stdin and duplicates the read end onto descriptor 0, then execs `wc -l`. Each child closes both original pipe descriptors after duplication. The parent closes both ends and waits for both specific children.

```
$ ./pipeline
112
$ ls /etc | wc -l
112
```

2.5.1 1. Comparison with the real shell

Both counts are 112 in the recorded environment. This equality, rather than the particular count, verifies the connection. `ls` writes one filename per line when its output is a pipe, and `wc -l` counts those lines.

2.5.2 2. Which child hangs after leaving a writer open

The experiment commented out only the parent's final `close(p[1])` in a temporary copy, compiled it and inspected it. The submitted `pipeline.c` remains the working version. The temporary program was still running after one second:

```
$ ./broken (parent write end left open)
PID  PPID STAT CMD
 8812  8776 Ss   /tmp/tmpeiiz8bal/broken
 8814  8812 S    wc -l
Still running after 1 second; stopped by test harness.
```

`ls` had already exited; `wc -l` was still sleeping. The test stopped the temporary process group after the observation, equivalent to interrupting the foreground pipeline with Ctrl-C.

2.5.3 3. What wc is waiting for

`wc` is waiting for EOF: after buffered bytes are consumed, read returns EOF only when every descriptor referring to the pipe's write end is closed. The parent's unused open writer keeps that condition false, while the parent waits for `wc` to exit. Pipe end closure is part of the communication protocol, not just resource tidiness.

2.6 Optional - Three stages

`pipeline3.c` implements `ls /usr/bin | grep x | wc -l` using two pipes and three children. The first child redirects stdout, the middle child redirects both stdin and stdout, and the last redirects stdin. Every child and the parent close all four original pipe descriptors after the required duplicates.

```
$ ./pipeline3
106
$ ls /usr/bin | grep x | wc -l
106
```

Both methods returned 106. The parent waits for every created child and reports a nonzero result if one fails. The optional source is included alongside the required programs.

3 Conclusions

The tests connect shell operations to fork, exec and wait. A zombie records an exited child until its parent reaps it. Redirection changes a descriptor in the child so an unchanged program writes to a file. Two- and three-stage pipelines match the shell counts, and the deliberately leaked write descriptor demonstrates that EOF depends on closing every writer.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 7: *Three Small Programs: Processes, Redirection, a Pipe*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, commit `f780d7850765d74cbdcd1fff4b822d8829f7c9bd`. Source checked on 23 October 2026. `https://github.com/mit-pdos/xv6-riscv/tree/f780d7850765d74cbdcd1fff4b822d8829f7c9bd`.

[3] Linux man-pages project. *fork(2), pipe(7), inode(7), path_resolution(7)*. `https://man7.org/linux/man-pages/`.

A Submitted source code

The following listings read the actual submitted source files, so the report and submission use the same implementation. Variant programs are included where relevant.

A.1 run.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command: ls -l */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

A.2 zombie.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command: ls -l */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    int status;
    sleep(20); /* Leave the exited child unreaped for observation. */
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

A.3 redirect.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        close(1);
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); _exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        _exit(127);
    }
    int status;
    if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); return 1; }
    if (!WIFEXITED(status)) return 1;
    if (WEXITSTATUS(status) != 0) return WEXITSTATUS(status);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

A.4 pipeline.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }
    pid_t producer = fork();
    if (producer < 0) { perror("fork"); close(p[0]); close(p[1]); return 1; }
    if (producer == 0) {
        close(1);
        if (dup(p[1]) < 0) { perror("dup"); _exit(1); }
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        _exit(127);
    }
    pid_t consumer = fork();
    if (consumer < 0) {
        perror("fork"); close(p[0]); close(p[1]);
        waitpid(producer, NULL, 0); return 1;
    }
    if (consumer == 0) {
        close(0);
        if (dup(p[0]) < 0) { perror("dup"); _exit(1); }
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        _exit(127);
    }
    close(p[0]); /* The parent uses neither end. */
    close(p[1]); /* Closing the last writer is necessary for EOF. */
    int a, b;
    if (waitpid(producer, &a, 0) < 0) { perror("waitpid"); return 1; }
    if (waitpid(consumer, &b, 0) < 0) { perror("waitpid"); return 1; }
    return !(WIFEXITED(a) && WEXITSTATUS(a) == 0 &&
             WIFEXITED(b) && WEXITSTATUS(b) == 0);
}
```

A.5 pipeline3.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int pipes[2][2];
    pid_t children[3];
    if (pipe(pipes[0]) < 0) { perror("pipe"); return 1; }
    if (pipe(pipes[1]) < 0) {
        perror("pipe"); close(pipes[0][0]); close(pipes[0][1]); return 1;
    }
    char *commands[3][4] = {
        {"ls", "/usr/bin", NULL, NULL},
        {"grep", "x", NULL, NULL},
        {"wc", "-l", NULL, NULL}
    };
    int count = 0, failed = 0;
    for (int stage = 0; stage < 3; stage++) {
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); failed = 1; break; }
        if (pid == 0) {
            if (stage > 0 && dup2(pipes[stage - 1][0], 0) < 0) {
                perror("dup2"); _exit(1);
            }
            if (stage < 2 && dup2(pipes[stage][1], 1) < 0) {
                perror("dup2"); _exit(1);
            }
            for (int i = 0; i < 2; i++) {
                close(pipes[i][0]); close(pipes[i][1]);
            }
            execvp(commands[stage][0], commands[stage]);
            perror("exec"); _exit(127);
        }
        children[count++] = pid;
    }
    for (int i = 0; i < 2; i++) {
        close(pipes[i][0]); close(pipes[i][1]);
    }
    for (int i = 0; i < count; i++) {
        int status;
        if (waitpid(children[i], &status, 0) < 0) { perror("waitpid"); failed = 1; }
        else if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) failed = 1;
    }
    return failed;
}
```